

Python Programming and Data Structures

Technical Notes

Rodrigo Kang

Table of contents

1	Introduction	2
1.1	Python as a Programming Language	3
2	Objects, Types, and Names	4
2.1	Dynamic Typing	6
2.2	Assignment and Aliasing	7
2.3	Identity and Equality	9
2.4	Mutability and Immutability	10
2.5	Copying Objects	12
3	Expressions and Operators	15
3.1	Boolean Expressions and Logical Operators	17
3.1.1	Truth Values	19
3.1.2	Short-Circuit Evaluation	20
4	Control Flow	21
4.1	Conditional Execution	22
4.2	Iteration	23
4.2.1	Iteration over an Iterable	24
4.2.2	Ranges and the Half-Open Convention	24
4.2.3	Condition-Controlled Iteration	26
4.2.4	Early Termination	27
5	Functions	28
5.1	Parameters and Arguments	29
5.2	Positional and Keyword Arguments	31
5.3	Default Arguments	32
5.4	Return Values	34
5.5	Scope and Namespaces	36
5.6	Functions as Objects	37
5.6.1	Anonymous Functions	38
5.7	Closures	39
5.8	Side Effects and Function Design	40

5.9	Documentation and Type Annotations	41
6	Numeric Types	43
6.1	Integers	43
6.1.1	Positional Representation	44
6.1.2	Integer Division and Remainders	45
6.2	Floating-Point Numbers	48
6.2.1	Binary Fractions	48
6.2.2	Floating-Point Representation	49
6.2.3	Representation Error	50
6.2.4	Comparing Floating-Point Values	51
6.2.5	Machine Precision	52
6.2.6	Accumulation of Rounding Error	53
6.3	Numerical Stability	54
6.4	Complex Numbers	55
7	Strings	57
7.1	String Literals	57
7.2	Strings as Sequences	58
7.2.1	Slicing	60
7.3	String Immutability	61
7.4	Concatenation and Repetition	62
7.5	Membership and Comparison	63
7.6	String Transformations	64
7.7	String Formatting	66
7.8	Unicode Text	67
7.9	Text and Bytes	68
8	References	69

1 Introduction

Programming provides a way to express computational processes in a form that a computer can execute. A computational problem begins with some input, applies a sequence of well-defined operations, and produces an output. The nature of the input and output depends on the problem: they may consist of numbers, text, images, measurements, records, or more complex structures.

An **algorithm** describes the finite sequence of steps used to perform this transformation. It specifies what must be done without necessarily prescribing how those steps are expressed in a particular programming language. A **program** is a concrete representation of one or more algorithms, together with the data and supporting logic required for their execution.

The relationship can be represented schematically as:

Input $\longrightarrow$ Algorithm $\longrightarrow$ Output.

For example, consider the problem of finding the largest measurement in a collection. The input is a sequence of measurements, the algorithm examines the values while maintaining the largest value encountered so far, and the output is the maximum measurement. The same algorithmic idea can be implemented in different programming languages and executed on different machines.

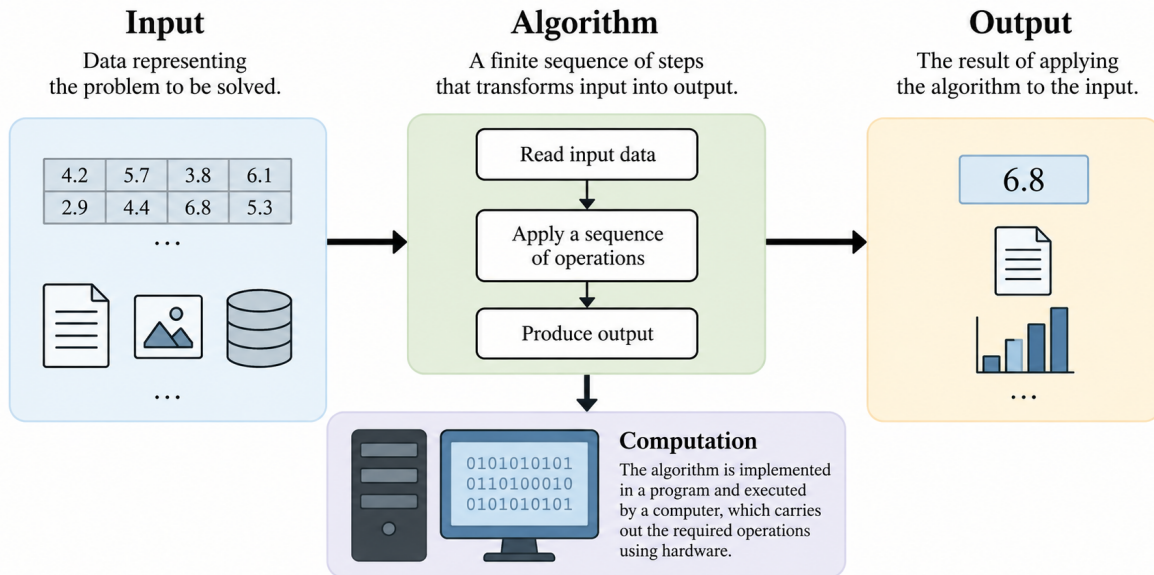


Figure 1: A computational process transforms input data into output through an algorithm implemented and executed by a computer.

Figure Figure 1 illustrates this general relationship. The algorithm provides the logical procedure, while computation refers to carrying out the operations described by that procedure. In practical computing systems, those operations are ultimately executed by hardware, although programming languages provide abstractions that allow programs to be expressed without working directly with machine instructions.

1.1 Python as a Programming Language

Python is a high-level, general-purpose programming language. It provides abstractions for representing data, expressing computations, organising programs, and building larger software systems. The language includes a relatively small core together with a large standard library and an extensive ecosystem of external packages.

Python programs are written as source code. A source file conventionally uses the .py extension and contains statements that can define objects, perform computations, control execution, define functions and classes, or interact with external resources.

For example:

```
1 measurements = [4.2, 5.7, 3.8, 6.1]
2 largest = max(measurements)
3
4 print(largest)
```

The source code describes the computation, but it is not executed directly by the processor in this form. Python implementations provide the machinery required to interpret the language and execute its instructions. This separation allows the programmer to work with concepts such as numbers, strings, functions, collections, and objects rather than processor instructions and memory addresses.

Python can be used interactively, where expressions and statements are evaluated as they are entered, or through scripts containing a sequence of instructions to be executed. Interactive execution is useful for exploration and experimentation, while scripts provide a persistent and reproducible representation of a computation. Both forms use the same language and follow the same underlying object model.

The usefulness of Python does not come only from its syntax. Much of its behaviour follows from a small number of underlying ideas: values are represented by objects, names refer to those objects, operations are defined by their types, and larger abstractions are constructed by combining objects and behaviours. Understanding these ideas is more important than memorising individual language constructs.

2 Objects, Types, and Names

A useful starting point for understanding Python is that computation operates on **objects**. Numbers, strings, lists, functions, classes, modules, and many other entities manipulated by a Python program are represented as objects.

Every object has three fundamental properties:

- an **identity**, which distinguishes it from other objects during its lifetime;
- a **type**, which determines the operations and behaviour supported by the object;
- a **value**, representing the information associated with the object.

Consider:

```
1 temperature = 21.5
```

It is tempting to describe this statement by saying that a variable called `temperature` contains the value 21.5. Although convenient in simple examples, that description becomes misleading once aliasing, mutability, function arguments, and container objects are considered.

A more accurate model is that Python creates or obtains an object representing the floating-point value 21.5 and binds the name `temperature` to that object:

`temperature` $\longrightarrow$ float object representing 21.5.

The assignment statement therefore establishes a relationship between a **name** and an **object**.

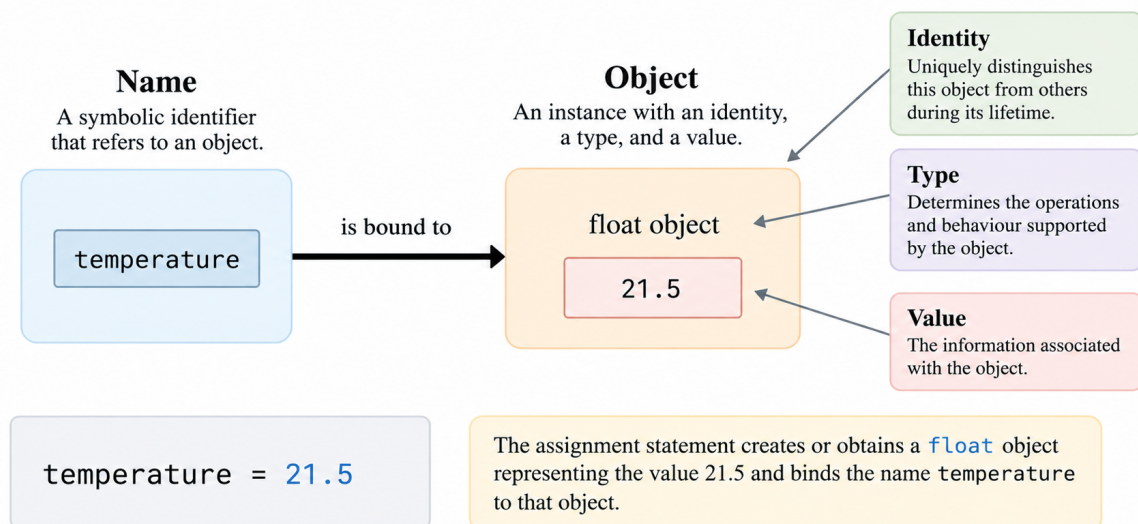


Figure 2: A Python name is bound to an object. The object has an identity, a type, and a value.

This distinction is fundamental because names and objects have different roles. A name belongs to a namespace and can later be bound to another object. The object exists independently of the particular name used to access it.

For example:

```
1 temperature = 21.5
2 temperature = 18.0
```

The second assignment does not modify the floating-point object representing 21.5. Instead, the name `temperature` is rebound to another object:

$$\begin{array}{c} \text{temperature} \longrightarrow 21.5 \\ \downarrow \\ \text{temperature} \longrightarrow 18.0. \end{array}$$

This behaviour is especially important because many Python objects are immutable: once created, their value cannot be changed. Reassigning a name that refers to an immutable object therefore changes the binding, not the object itself.

2.1 Dynamic Typing

The type belongs to the object rather than to the name. This is the basis of Python's **dynamic typing**.

A name can therefore refer to objects of different types at different points during execution:

```
1 result = 42
2 result = 3.14
3 result = "complete"
```

The name `result` has not changed its own type. Rather, it has successively been bound to an int, a float, and a str object.

The type of an object can be inspected with `type()`:

```
1 value = 42
2
3 type(value)
```

which reports `int`.

Types are important because they determine the operations that an object supports and the meaning of those operations. Addition, for example, has different behaviour depending on the operands:

```
1 2 + 3
2 "data" + "base"
```

The first expression performs numerical addition, while the second performs string concatenation. The `+` operator does not have a single meaning independent of its operands; its behaviour is determined by their types.

Python provides several fundamental built-in types. Among those used throughout these notes are:

Type	Purpose	Mutable
<code>bool</code>	Boolean values	No
<code>int</code>	Integer numbers	No
<code>float</code>	Floating-point numbers	No
<code>complex</code>	Complex numbers	No
<code>str</code>	Text	No
<code>list</code>	Mutable sequence	Yes
<code>tuple</code>	Immutable sequence	No
<code>dict</code>	Key-value mapping	Yes
<code>set</code>	Set of distinct objects	Yes
<code>frozenset</code>	Immutable set	No

The distinction between mutable and immutable objects will become important throughout the discussion of Python. It affects assignment, function calls, copying, hashing, and the behaviour of data structures.

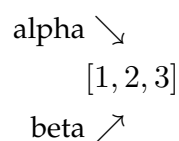
2.2 Assignment and Aliasing

Because assignment binds names to objects, more than one name can refer to the same object. This situation is known as **aliasing**.

Consider:

```
1 alpha = [1, 2, 3]
2 beta = alpha
```

After the second assignment, no second list has been created. Both names refer to the same list:



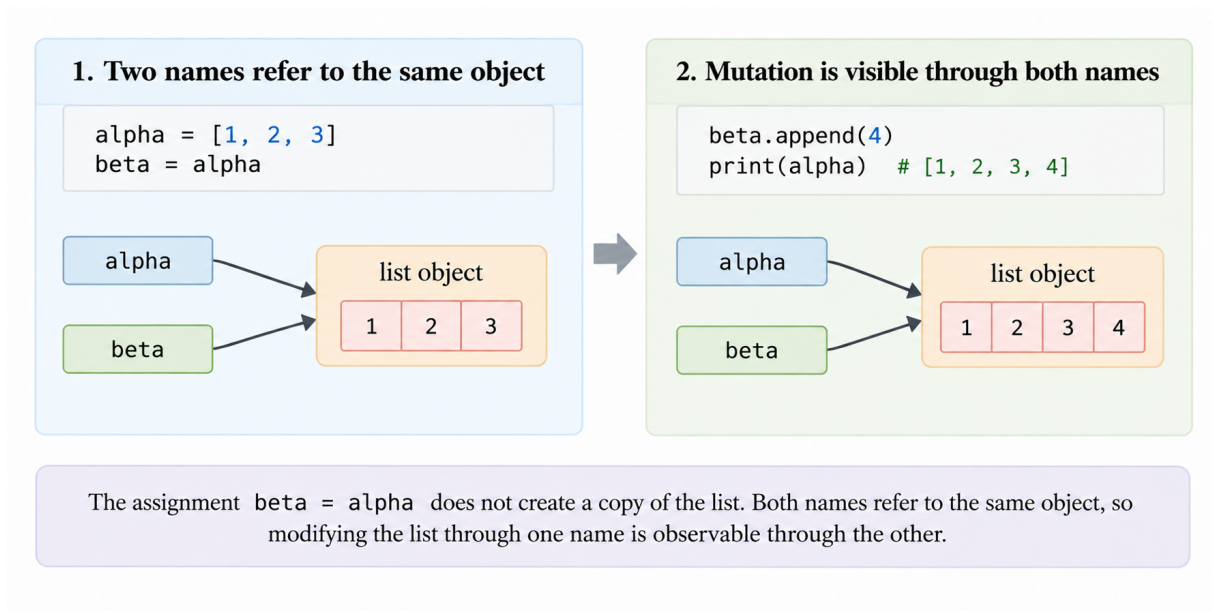


Figure 3: Two Python names may refer to the same object. Mutating that object is therefore observable through either name.

The consequence becomes visible when the object is modified:

```
1 beta.append(4)
2
3 print(alpha)
```

which produces:

```
1 [1, 2, 3, 4]
```

The assignment to `beta` did not copy the list. Since `alpha` and `beta` refer to the same mutable object, a mutation performed through one name is observable through the other.

This must be distinguished from **rebinding**:

```
1 beta = beta + [5]
```

The expression on the right creates a new list, after which `beta` is rebound to that new object. The original list remains associated with `alpha`.

The difference is therefore:

mutation changes an object	whereas	assignment changes a binding.
----------------------------	---------	-------------------------------

This distinction is one of the central ideas in Python's object model. It will reappear when considering mutable containers, function arguments, copying, class instances, and data structures.

2.3 Identity and Equality

Aliasing introduces an important distinction between two different questions that can be asked about objects:

1. Are two names referring to the same object?
2. Do two objects represent equivalent values?

Python provides different operators for these questions. The operators `is` and `is not` compare **identity**, while `==` and `!=` compare **equality**.

Consider two independently created lists:

```
1 alpha = [1, 2, 3]
2 beta = [1, 2, 3]
```

The lists contain the same values, so:

```
1 alpha == beta
```

evaluates to `True`. However:

```
1 alpha is beta
```

evaluates to `False`, because `alpha` and `beta` refer to two distinct list objects.

The distinction can be expressed as:

identity $\neq$ equality.

Identity concerns the object itself. Equality concerns whether two objects are considered equivalent according to the semantics of their types.

The situation changes if one name is assigned directly to the other:

```
1 alpha = [1, 2, 3]
2 beta = alpha
```

Now:

```
1 alpha is beta
```

evaluates to True, because both names refer to the same object. Since an object is also equal to itself under the usual equality semantics, `alpha == beta` is also True.

The two cases can therefore be distinguished conceptually as:

$$\begin{aligned}\text{alpha} == \text{beta} &\rightarrow \text{equivalent values,} \\ \text{alpha is beta} &\rightarrow \text{same object.}\end{aligned}$$

For most value comparisons, equality is the relevant relation. Identity testing is appropriate when the program specifically needs to determine whether two references designate the same object. A common example is comparison with None:

```
1 if result is None:
2     ...
```

Here the intention is not to ask whether result has a value equivalent to None, but whether it refers to the particular object used by Python to represent the absence of a value.

This distinction becomes even more important with user-defined classes. The meaning of equality can be defined as part of the behaviour of a type, whereas identity remains a property of the individual object. Equality will therefore be revisited when the Python object model and special methods are considered in greater detail.

2.4 Mutability and Immutability

The effect of aliasing depends strongly on whether the referenced object is **mutable** or **immutable**.

A mutable object can change its state after it has been created. Lists, dictionaries, and sets are common examples. An immutable object cannot change its value after creation. Numbers, strings, tuples, and frozensets are examples of immutable built-in types.

Consider a list:

```
1 values = [10, 20, 30]
```

The operation:

```
1 values.append(40)
```

changes the existing list object. The binding of the name `values` does not need to change:

$$\text{values} \longrightarrow [10, 20, 30]$$

becomes:

$$\text{values} \longrightarrow [10, 20, 30, 40].$$

The object has been mutated.

Now consider an integer:

```
1 count = 10
2 count = count + 1
```

An integer object cannot be changed from 10 into 11. The expression on the right-hand side is evaluated first, producing an integer object representing 11. The name `count` is then rebound to that object.

Conceptually:

$$\text{count} \longrightarrow 10$$

is replaced by:

$$\text{count} \longrightarrow 11.$$

The integer object representing 10 has not been modified.

This distinction explains why superficially similar operations may have different consequences when aliases exist.

For example:

```
1 alpha = [1, 2]
2 beta = alpha
3
4 alpha.append(3)
```

Since the list is mutable, both names still refer to the modified object:

```
1 alpha -> [1, 2, 3]
2 beta -> [1, 2, 3]
```

By contrast:

```
1 alpha = 10
2 beta = alpha
3
4 alpha = alpha + 1
```

does not alter the object to which beta refers. Instead, alpha is rebound:

```
1 alpha -> 11
2 beta -> 10
```

Mutability is therefore not simply a property used to classify built-in types. It has direct consequences for how state changes propagate through a program.

An important subtlety is that the immutability of a container does not imply that every object referenced by the container is itself immutable. A tuple, for example, cannot have its references replaced after creation, but it may contain a mutable object:

```
1 record = ("sensor-01", [21.4, 21.7])
2 record[1].append(22.0)
```

The tuple itself has not been modified: it still refers to the same string and the same list. The list contained within it, however, is mutable and has changed.

Thus, immutability concerns the state of the object under consideration, not necessarily the state of every object reachable from it.

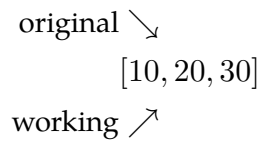
2.5 Copying Objects

Aliasing is useful when several parts of a program are intended to operate on the same object. In other situations, independent state is required.

Suppose an existing collection is used as the starting point for another collection:

```
1 original = [10, 20, 30]
2 working = original
```

This does not create an independent list. The assignment creates another reference to the same object:



Consequently:

```
1 working.append(40)
```

also changes what is observed through original.

If an independent outer list is required, a new list can be constructed:

```
1 working = list(original)
```

or, equivalently for a list:

```
1 working = original.copy()
```

Now original and working refer to distinct list objects. Adding or removing an element from one list does not change the other.

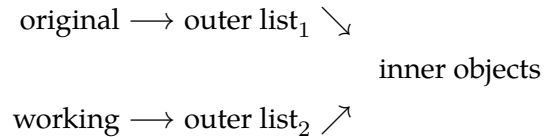
For flat collections containing immutable elements, this distinction is often sufficient. Nested mutable structures reveal a deeper issue.

Consider:

```
1 original = [  
2     ["A", 10],  
3     ["B", 20],  
4 ]  
5  
6 working = original.copy()
```

Two list objects now exist at the outer level, but their elements still refer to the same inner lists.

The structure is conceptually:



This is a **shallow copy**. The outer container has been duplicated, but the objects referenced by that container have not.

Therefore:

```
1 working[0][1] = 99
```

also changes:

```
1 original[0]
```

because:

```
1 working[0] is original[0]
```

is True.

When the nested objects must also be independent, a **deep copy** may be required. Python provides this operation through the copy module:

```
1 import copy
2
3 working = copy.deepcopy(original)
```

A deep copy recursively constructs independent copies of the objects contained within the structure rather than merely duplicating the outer container.

The three situations can be summarised as:

assignment → same object,
shallow copy → new outer object, shared nested objects,
deep copy → independent copied structure.

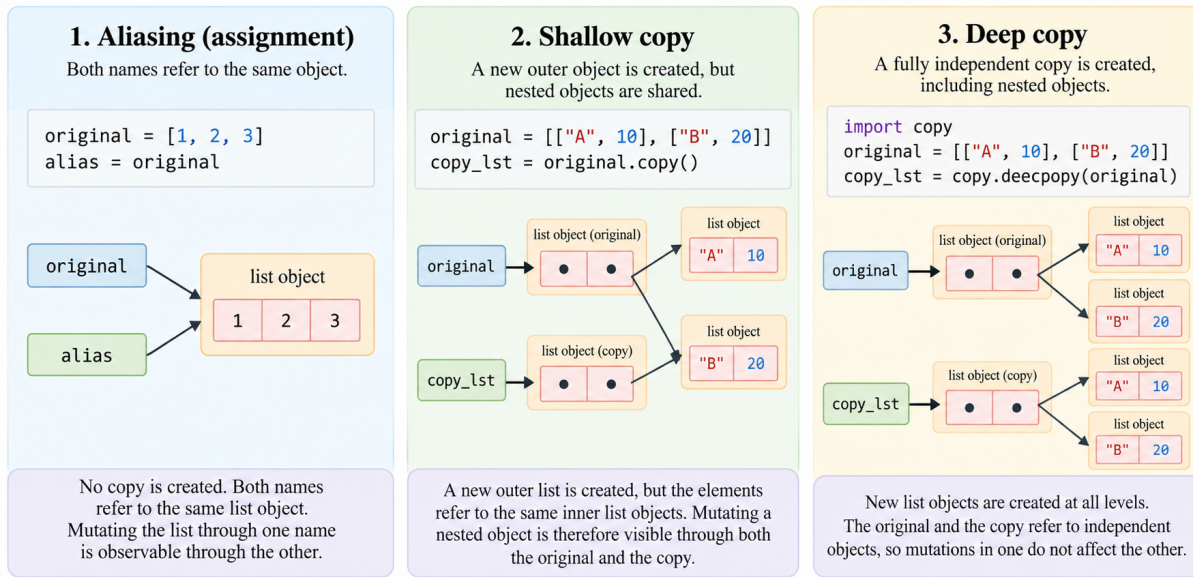


Figure 4: Aliasing, shallow copying, and deep copying differ in which objects are shared between the original structure and its copy.

Deep copying should not be understood as a universally preferable form of copying. Sharing objects can be intentional and efficient, while duplicating an entire object structure may be unnecessary or inappropriate. The relevant question is whether the state represented by the objects is intended to be shared.

The distinction between references and copies will become particularly important when working with nested containers and data structures. Many Python containers are **referential structures**: they store references to objects rather than embedding independent copies of those objects. Understanding what is copied and what remains shared is therefore necessary for reasoning correctly about their behaviour.

3 Expressions and Operators

Objects become useful in a computation through the operations that can be performed on them. An **expression** combines objects, names, literals, operators, and function calls to produce a value.

For example:

```
1 subtotal = price * quantity
```

contains the expression:

```
1 price * quantity
```

which is evaluated before the resulting object is bound to the name subtotal.

The meaning of an operator depends on the types of its operands. The expression:

```
1 2 + 3
```

performs numerical addition, while:

```
1 "data" + "science"
```

performs string concatenation.

This is not merely syntactic convenience. Operators are part of the behaviour provided by types. Python uses a common notation for operations that may have different meanings for different kinds of objects.

Expressions may combine several operations:

```
1 result = a + b * c
```

In such cases, Python applies a defined operator precedence. Multiplication is evaluated before addition, so the expression is interpreted as:

```
1 result = a + (b * c)
```

rather than:

```
1 result = (a + b) * c
```

Explicit parentheses should be used when they clarify the intended grouping, even when the default precedence would produce the same result.

Operators can broadly express arithmetic, comparison, identity, membership, and logical relationships. Their detailed behaviour will be introduced together with the types and control structures for which they are relevant rather than treated as an isolated catalogue of syntax.

This is particularly useful for Boolean expressions. Conditions such as:

```
1 temperature > 30
```

or:

```
1 value is None
```

do not merely perform computations; they produce Boolean results that can determine which parts of a program are executed. This provides the connection between expressions and **control flow**, which determines how execution proceeds through a program.

3.1 Boolean Expressions and Logical Operators

Many computations require decisions rather than only the production of numerical or textual values. A program may need to determine whether a measurement exceeds a threshold, whether a collection contains an element, or whether several conditions hold simultaneously.

These decisions are expressed through Boolean values. Python provides the type `bool`, whose two possible values are:

```
1 True
2 False
```

Comparison operators produce Boolean results. For example:

```
1 temperature = 28.5
2
3 temperature > 25
```

evaluates to `True`, while:

```
1 temperature < 20
```

evaluates to `False`.

The usual ordering comparisons are:

Operator	Meaning
$a < b$	a is less than b
$a \leq b$	a is less than or equal to b

Operator	Meaning
$a > b$	a is greater than b
$a \geq b$	a is greater than or equal to b
$a == b$	a and b are equal
$a != b$	a and b are not equal

As discussed previously, `is` and `is not` answer a different question: they compare object identity rather than value equality.

Python also supports chained comparisons. A mathematical condition such as:

$$0 \leq x < 1$$

can be written directly as:

```
1 0 <= x < 1
```

This expresses the intended relationship more naturally than writing the two comparisons separately.

More complex conditions can be constructed with the logical operators `not`, `and`, and `or`.

For propositions P and Q , their behaviour can be summarised by the following truth table:

P	Q	P and Q	P or Q
False	False	False	False
False	True	False	True
True	False	False	True
True	True	True	True

Negation reverses the truth value:

P	not P
False	True
True	False

These operators allow conditions to represent the actual logical requirements of a problem. For example:

```
1 temperature > 20 and temperature < 30
```

requires both comparisons to hold, while:

```
1 status == "warning" or status == "critical"
```

requires at least one of them.

Logical expressions should generally be written to make the underlying condition evident. Parentheses can be useful when several operators are combined, even when Python's precedence rules would make them unnecessary:

```
1 (is_valid and has_data) or use_default
```

The purpose of such expressions is not merely to produce True or False. They provide the logical basis for controlling which computations a program performs.

3.1.1 Truth Values

Conditions in Python do not always require an explicit bool object. Objects of other types can be evaluated in a Boolean context.

Several common values are considered false, including numeric zero, None, and empty containers:

```
1 0
2 0.0
3 ""
4 []
5 {}
6 set()
7 None
```

Nonzero numbers and nonempty containers are generally considered true.

This allows conditions such as:

```
1 records = [...]
2
3 if records:
4     ...
```

rather than:

```
1 if len(records) > 0:
2     ...
```

The first form expresses the relevant question directly: whether records contains data.

Truth-value testing should nevertheless be used when its meaning is clear. Different states may need to be distinguished explicitly. For example, an empty collection and None may both evaluate as false while representing different situations in a program.

3.1.2 Short-Circuit Evaluation

The operators and or are evaluated from left to right and use **short-circuit evaluation**. Python evaluates the second operand only when it is necessary to determine the result.

For:

```
1 P and Q
```

if P is false, the complete expression cannot be true, regardless of Q. Therefore Q does not need to be evaluated.

Similarly, for:

```
1 P or Q
```

if P is true, the complete expression is already true and Q does not need to be evaluated.

This behaviour has practical consequences. Consider a search through a sequence:

```
1 index = 0
2
3 while index < len(data) and data[index] != target:
4     index += 1
```

The first condition verifies that index is valid before the expression attempts to access:

```
1 data[index]
```

When `index == len(data)`, the first condition is false. Because `and` short-circuits, the second condition is not evaluated, preventing an invalid access beyond the end of the sequence.

The order of the conditions is therefore significant. Writing:

```
1 while data[index] != target and index < len(data):  
2     ...
```

would not provide the same protection, because `data[index]` would be evaluated first.

Short-circuit evaluation is consequently more than an optimisation. It can be part of the correctness of an expression.

4 Control Flow

A program consisting only of expressions evaluated sequentially would follow the same path every time it executes. Most computations require greater flexibility: some operations should occur only under certain conditions, while others must be repeated until a task is complete.

Control flow determines the order in which statements are executed.

Python provides two fundamental mechanisms for changing sequential execution:

- **selection**, which chooses among alternative paths;
- **iteration**, which repeats a computation.

These mechanisms are expressed primarily through conditional statements and loops.

Python uses indentation to define blocks of statements. Indentation is therefore part of the structure of the language rather than merely a formatting convention.

For example:

```
1 if temperature > 30:  
2     alert = True  
3     message = "High temperature"  
4  
5 log_temperature(temperature)
```

The two indented statements belong to the conditional block. The final statement lies outside it and is executed regardless of the condition.

4.1 Conditional Execution

An if statement allows execution to depend on a condition evaluated at runtime.

Consider a simple classification:

```
1 if temperature > 30:
2     status = "high"
3 else:
4     status = "normal"
```

Exactly one of the two assignments is performed. The selected path depends on the Boolean value of:

```
1 temperature > 30
```

More than two mutually exclusive cases can be represented with elif:

```
1 if temperature < 10:
2     status = "low"
3 elif temperature <= 30:
4     status = "normal"
5 else:
6     status = "high"
```

The conditions are evaluated in order. Once one evaluates to true, its corresponding block is executed and the remaining alternatives are skipped.

This ordering matters whenever conditions overlap. Consider:

```
1 if score >= 50:
2     grade = "pass"
3 elif score >= 80:
4     grade = "distinction"
```

A score of 90 satisfies the first condition, so the second condition is never reached. The intended classification requires the more restrictive condition first:

```
1 if score >= 80:
2     grade = "distinction"
3 elif score >= 50:
4     grade = "pass"
5 else:
6     grade = "fail"
```

Conditional logic therefore involves more than writing syntactically valid comparisons. The branches must partition the relevant cases in a way that represents the problem correctly.

Several independent if statements should also be distinguished from a single if–elif–else chain. With independent statements:

```
1 if condition_a:
2     ...
3
4 if condition_b:
5     ...
```

both blocks may execute. In an if–elif chain, at most one branch is selected.

This difference reflects two different logical structures: independent conditions versus mutually exclusive alternatives.

4.2 Iteration

Many computational tasks involve applying an operation repeatedly. A collection may need to be processed element by element, a search may continue until a target is found, or a numerical method may iterate until a convergence criterion is satisfied.

Python provides two principal loop constructs:

```
1 for
2 while
```

They both express repetition, but they correspond to different ways of describing when repetition should continue.

4.2.1 Iteration over an Iterable

A for loop consumes values from an **iterable**:

```
1 measurements = [4.2, 5.7, 3.8, 6.1]
2
3 for value in measurements:
4     print(value)
```

The important idea is that Python's for loop is fundamentally an iteration over values, not a counter-based loop. During each iteration, the loop variable is bound to the next object produced by the iterable.

For example, a maximum can be computed as:

```
1 measurements = [4.2, 5.7, 3.8, 6.1]
2
3 largest = measurements[0]
4
5 for value in measurements[1:]:
6     if value > largest:
7         largest = value
```

The loop directly expresses the operation being performed: examine each remaining measurement and update the current maximum when a larger value is encountered.

This pattern appears throughout algorithms:

initial state → process next element → update state → repeat.

The variable `largest` represents the state accumulated by the computation. After each iteration, it contains the largest value encountered so far.

4.2.2 Ranges and the Half-Open Convention

Some computations require positions rather than only values. Python provides `range` for generating sequences of integers suitable for iteration.

The expression:

```
1 range(n)
```

represents the integers:

$$0, 1, 2, \dots, n - 1.$$

The upper bound n is excluded. More generally:

```
1 range(start, stop)
```

follows the half-open interval:

$$[\text{start}, \text{stop}).$$

This convention is particularly useful for indexing. A sequence of length n has valid positions:

$$0, 1, \dots, n - 1,$$

which correspond exactly to:

```
1 range(n)
```

For example:

```
1 for index in range(len(data)):
2     print(index, data[index])
```

There is, however, no reason to introduce an index when only the values are needed. If the position and value are both relevant, Python provides `enumerate`:

```
1 for index, value in enumerate(data):
2     print(index, value)
```

This expresses the intention more directly than manually managing an index.

4.2.3 Condition-Controlled Iteration

A while loop is appropriate when repetition is governed by a condition rather than by consuming a predefined iterable:

```
1 while condition:
2     ...
```

The condition is evaluated before every iteration. Execution continues while it remains true.

Consider an iterative approximation in which successive estimates are computed until the change becomes sufficiently small:

```
1 estimate = initial_estimate
2
3 while error > tolerance:
4     estimate = update(estimate)
5     error = compute_error(estimate)
```

The number of iterations is not necessarily known beforehand. Termination depends on the evolving state of the computation.

This distinguishes the two principal loop forms conceptually:

for → iterate over values from an iterable,
while → repeat while a condition holds.

A while loop must eventually make progress toward making its condition false unless nontermination is intentional. Failure to update the relevant state produces an infinite loop.

For example:

```
1 index = 0
2
3 while index < len(data):
4     process(data[index])
5     index += 1
```

terminates because index increases on each iteration and eventually reaches len(data).

This idea of **progress toward termination** becomes important when reasoning about algorithm correctness. A loop should not only perform the intended operation during each iteration; there must also be a reason why the process eventually stops.

4.2.4 Early Termination

Sometimes a loop does not need to examine every possible element. Once the required information has been obtained, continuing would perform unnecessary work.

A search is a typical example:

```
1 found = False
2
3 for value in data:
4     if value == target:
5         found = True
6         break
```

The `break` statement terminates the nearest enclosing loop immediately.

The `continue` statement has a different purpose. It terminates only the current iteration and proceeds with the next one:

```
1 for value in data:
2     if value < 0:
3         continue
4
5     process(value)
```

Here negative values are skipped while iteration continues over the remaining elements.

Both constructs can simplify control flow when used deliberately. Excessive use, however, can make the execution path difficult to follow. A loop should generally retain a clear termination condition and a clear purpose.

Python also allows a loop to have an `else` clause. The `else` block executes when the loop finishes normally rather than through `break`.

This is useful for searches:

```
1 for value in data:
2     if value == target:
3         print("Target found")
4         break
5 else:
6     print("Target not found")
```

The else clause expresses a precise condition: the loop exhausted its iterable without encountering a break.

This behaviour can initially appear unusual, but it avoids maintaining a separate Boolean flag when the relevant distinction is simply whether the search terminated early.

Control structures provide the machinery for expressing algorithmic processes, but larger programs also require a way to isolate computations behind well-defined interfaces. This is the role of **functions**, which allow behaviour to be named, parameterised, reused, and composed.

5 Functions

As programs grow, repeatedly expressing the same computation becomes difficult to maintain and reason about. More importantly, many computations represent meaningful operations that should be understood independently of the details required to perform them.

A **function** provides a named abstraction for such a computation. It defines an interface through which information is supplied, performs a particular operation, and may return a result.

Consider the task of computing the mean of a collection of measurements. Without a function, the calculation could be written directly wherever it is needed:

```
1 mean = sum(measurements) / len(measurements)
```

If the same operation is required in several parts of a program, however, repeating the expression also repeats knowledge about how the computation is performed. A function gives that operation a name:

```
1 def mean(values):  
2     return sum(values) / len(values)
```

The caller can then use:

```
1 average = mean(measurements)
```

without depending on the internal steps of the calculation.

This separation establishes a simple interface:

arguments $\longrightarrow$ function $\longrightarrow$ return value.

A useful function therefore does more than avoid duplicated code. It isolates a coherent operation behind a defined contract: what information the function expects, what it computes, what it returns, and whether it changes any externally visible state.

5.1 Parameters and Arguments

The names declared in a function definition are its **parameters**:

```
1 def scale(value, factor):  
2     return value * factor
```

When the function is called:

```
1 result = scale(12, 0.5)
```

the objects supplied by the caller are its **arguments**. In this call, 12 is associated with the parameter value, and 0.5 with factor.

This distinction is useful, but the more important question is how information is passed into a function.

Python's parameter passing follows the same object-and-binding model introduced earlier. A function call creates local parameter names and binds them to the objects supplied by the caller.

Suppose:

```
1 measurements = [4.2, 5.7, 3.8]
```

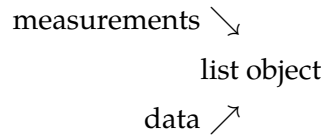
and a function is called as:

```
1 process(measurements)
```

for a definition:

```
1 def process(data):  
2     ...
```

During the call, the local parameter `data` refers to the same list object as `measurements`:



The list is not copied merely because it was passed to a function.

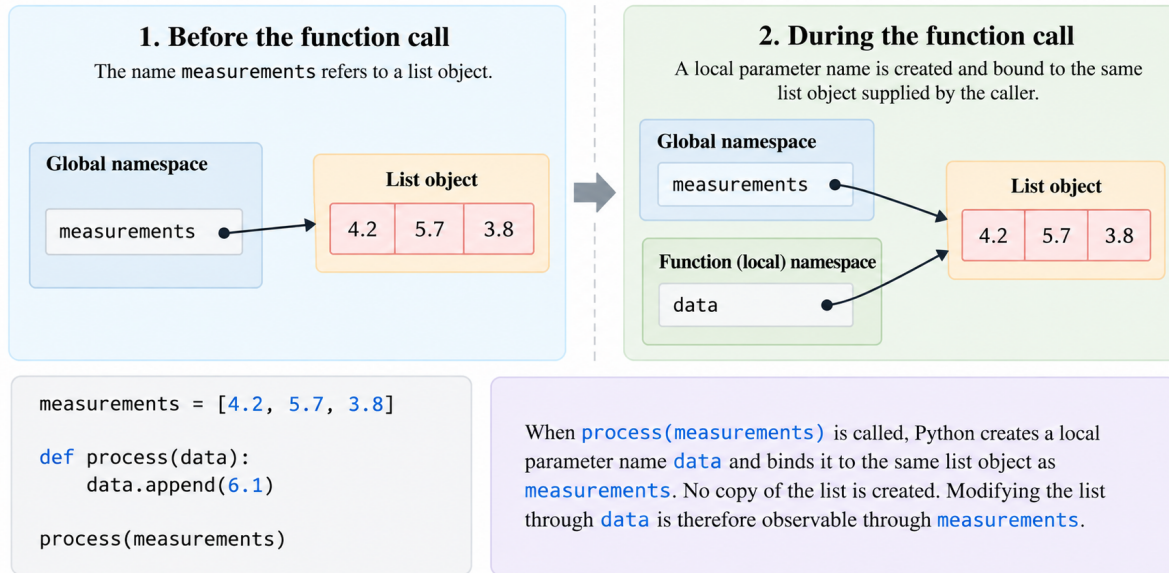


Figure 5: When an object is passed to a Python function, the local parameter name is bound to the same object supplied by the caller.

This model is more precise than describing Python simply as either “pass by value” or “pass by reference”. The essential mechanism is **object binding**: the function receives references to objects, and its parameter names are local bindings to those objects.

The consequences follow directly from the distinction between mutation and rebinding.

Consider:

```
1 def add_measurement(data, value):
2     data.append(value)
```

Calling:

```
1 measurements = [4.2, 5.7]
2 add_measurement(measurements, 3.8)
```

modifies the original list:

```
1 [4.2, 5.7, 3.8]
```

because data and measurements refer to the same mutable object.

By contrast:

```
1 def replace_data(data):  
2     data = [0.0, 0.0]
```

followed by:

```
1 measurements = [4.2, 5.7]  
2 replace_data(measurements)
```

does not replace the caller's list. The assignment changes only the local binding data.

The distinction is therefore the same one encountered earlier:

mutation can affect a shared object, whereas rebinding a parameter is local to the function.

This behaviour is not a special rule for function arguments. It follows directly from Python's general assignment semantics.

5.2 Positional and Keyword Arguments

Arguments can be associated with parameters by position:

```
1 def convert_temperature(value, scale):  
2     ...
```

```
1 convert_temperature(20, "C")
```

Here the first argument is bound to value and the second to scale.

Python also allows arguments to be supplied by parameter name:

```
1 convert_temperature(value=20, scale="C")
```

These are **keyword arguments**. They can make a call clearer when the meaning of a value would otherwise be ambiguous.

For example:

```
1 compute_interval(10, 0.95, True)
```

reveals little about the meaning of the final two arguments, while:

```
1 compute_interval(  
2     sample_size=10,  
3     confidence=0.95,  
4     corrected=True,  
5 )
```

makes the interface considerably clearer.

Keyword arguments also allow optional parameters to be specified selectively when defaults are available.

Functions may accept a variable number of positional or keyword arguments through `*args` and `**kwargs`. Conceptually, these collect additional arguments into a tuple and a dictionary, respectively:

```
1 def function(*args, **kwargs):  
2     ...
```

This mechanism is useful for interfaces whose number of inputs is genuinely variable or for forwarding arguments to another function. It should not be used merely to avoid defining a clear interface when the expected parameters are known.

5.3 Default Arguments

A parameter can provide a default value:

```
1 def power(base, exponent=2):  
2     return base ** exponent
```

The function can then be called as:

```
1 power(4)
```

which uses the default value 2, or as:

```
1 power(4, 3)
```

which explicitly supplies the exponent.

Defaults are useful when one behaviour is common but alternative behaviour should remain available.

There is, however, an important consequence of Python's object model: **default argument objects are created when the function definition is evaluated, not each time the function is called.**

Consider:

```
1 def collect(value, items=[]):
2     items.append(value)
3     return items
```

A first call:

```
1 collect(1)
```

returns:

```
1 [1]
```

but a subsequent call:

```
1 collect(2)
```

returns:

```
1 [1, 2]
```

rather than a new list containing only 2.

The reason is that the default list is a single mutable object associated with the function definition. Calls that omit items repeatedly use that same object:

function definition $\longrightarrow$ one default list object.

The usual pattern for an independently created mutable value is therefore:

```
1 def collect(value, items=None):
2     if items is None:
3         items = []
4
5     items.append(value)
6     return items
```

Now a new list is created during each call in which no collection is supplied.

This example brings together several earlier concepts:

object lifetime + mutability + aliasing + function definition semantics.

The issue is not that lists are unsuitable as function arguments. The issue is the unintended persistence of a mutable default object between calls.

5.4 Return Values

A function communicates a result to its caller with return:

```
1 def square(value):
2     return value * value
```

Executing return immediately terminates the current function call and provides the referenced object to the caller.

The returned object can then be bound to a name:

```
1 result = square(5)
```

Conceptually, the result of the function call participates in assignment like any other expression:

result $\longrightarrow$ object returned by square(5).

A function does not need an explicit return statement. If execution reaches the end of the function without one, Python returns None.

For example:

```
1 def display(value):  
2     print(value)
```

Calling:

```
1 result = display(10)
```

prints 10, but result is bound to None.

This distinction separates two different effects:

```
1 print(value)
```

produces output, whereas:

```
1 return value
```

provides an object to the caller.

Confusing these operations often leads to functions that are unnecessarily difficult to reuse. A function that computes a value can usually be composed more easily when it returns that value and leaves presentation to another part of the program.

Python functions can appear to return several values:

```
1 def bounds(values):  
2     return min(values), max(values)
```

However, this does not return two independent objects through two separate channels. The comma constructs a tuple, so the function returns one tuple object:

```
1 limits = bounds(data)
```

The common form:

```
1 lower, upper = bounds(data)
```

works because the returned tuple is subsequently unpacked into two names.

5.5 Scope and Namespaces

Functions introduce local names, which raises another question: when a name appears in an expression, how does Python determine which object it refers to?

A **namespace** associates names with objects. A **scope** determines the region of a program in which a particular namespace is directly relevant for name resolution.

Consider:

```
1 threshold = 10
2
3 def classify(value):
4     limit = threshold
5
6     if value > limit:
7         return "high"
8
9     return "normal"
```

During a call to `classify`, the names `value` and `limit` belong to the function's local scope. The name `threshold`, however, is not defined locally, so Python searches an enclosing namespace and finds the global binding.

Name resolution follows nested scopes. A useful description is the **LEGB** order:

Local → Enclosing → Global → Built-in.
--

The local scope belongs to the current function call. Enclosing scopes arise from lexically enclosing functions. Global scope corresponds to the module, while the built-in namespace contains names provided by Python such as `len`, `sum`, and `print`.

If no binding can be found through the relevant scopes, name resolution fails with a `NameError`.

Local bindings are independent of bindings with the same name in another scope:

```

1 value = 100
2
3 def transform():
4     value = 10
5     return value

```

The assignment inside `transform` creates a local binding. It does not change the global binding:

```

1 transform() # 10
2 value      # 100

```

This separation is important for modular reasoning. A function should normally be understandable from its parameters, local state, and clearly identified external dependencies rather than relying on uncontrolled global state.

Python provides `global` and `nonlocal` when rebinding a name from another scope is genuinely required. Their existence is important for understanding the language, but routine function design should not depend on them unnecessarily. Explicit inputs and outputs generally make data flow easier to understand.

5.6 Functions as Objects

Functions themselves are objects in Python. A function definition creates a function object and binds a name to it.

For example:

```

1 def square(value):
2     return value * value

```

establishes a relationship conceptually similar to:

$$\text{square} \longrightarrow \text{function object.}$$

Because functions are **first-class objects**, they can be assigned to other names:

```

1 operation = square
2
3 operation(5)

```

They can also be stored in data structures, passed as arguments, and returned by other functions.

This allows behaviour itself to become part of a program's data flow.

A simple example is sorting records according to a particular component:

```
1 records = [  
2     ("A", 3.2),  
3     ("B", 2.7),  
4     ("C", 4.1),  
5 ]  
6  
7 def measurement(record):  
8     return record[1]  
9  
10 ordered = sorted(records, key=measurement)
```

The sorted function receives another function through its key parameter. For each record, that function determines the value used for ordering.

The important abstraction is:

data + behaviour supplied as an argument.

A function that receives another function or returns one is commonly called a **higher-order function**.

This mechanism is widely useful because an algorithm can remain fixed while part of its behaviour is supplied by the caller. Sorting, for example, does not need a separate implementation for every possible interpretation of ordering.

5.6.1 Anonymous Functions

For short operations that do not need a reusable name, Python provides lambda expressions.

The previous key function could be written as:

```
1 ordered = sorted(records, key=lambda record: record[1])
```

A lambda creates a function object, but its body is restricted to a single expression.

This makes it useful for small, local transformations. It is not inherently preferable to a named function. When behaviour has a meaningful role, requires documentation, or becomes difficult to read as a single expression, a regular function definition is clearer.

5.7 Closures

First-class functions become particularly useful when combined with lexical scope.

Consider a function that constructs another function:

```
1 def make_multiplier(factor):
2     def multiply(value):
3         return value * factor
4
5     return multiply
```

Calling:

```
1 double = make_multiplier(2)
```

returns the inner function. The call to `make_multiplier` has already finished, but:

```
1 double(5)
```

still produces:

```
1 10
```

The returned function retains access to the binding of `factor` from the enclosing scope in which it was defined.

This combination of a function and its retained lexical environment is known as a **closure**.

Conceptually:

closure = function + captured environment.

Closures allow behaviour to be configured without introducing global state. In the previous example, `make_multiplier(2)` and `make_multiplier(3)` can create independent functions whose behaviour depends on different captured values.

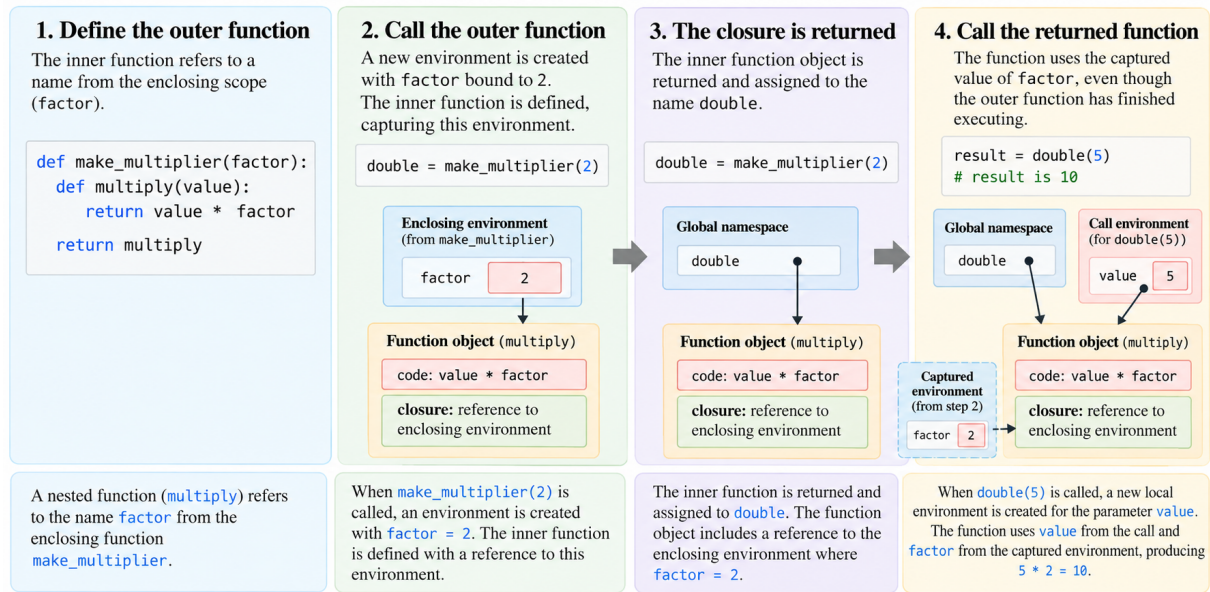


Figure 6: A closure retains access to bindings from the enclosing lexical scope even after the enclosing function has returned.

If an inner function must rebind a variable in an enclosing function rather than merely read it, Python provides the nonlocal statement. This is useful in some stateful closures, although explicit objects may become clearer when the amount of retained state or behaviour grows substantially.

5.8 Side Effects and Function Design

A function may return a value, modify an existing object, perform input or output, or combine several of these behaviours. Operations that affect state outside the function's local computation are commonly called **side effects**.

For example:

```
1 def square(value):
2     return value * value
```

computes a result without modifying its argument or producing external output.

By contrast:

```
1 def normalise_in_place(values):
2     ...
```

may deliberately modify the supplied collection, while:

```
1 def write_report(data, path):  
2     ...
```

necessarily interacts with an external resource.

Side effects are not inherently undesirable. Mutation, file operations, database access, logging, and user interaction are necessary in real programs. What matters is that the behaviour of a function is deliberate and visible through its interface and documentation.

A useful function generally has a coherent responsibility. Its inputs should be explicit, its return value should have a clear meaning, and mutations or other side effects should not be surprising.

For example, a function named:

```
1 compute_mean(values)
```

would be unexpected if it also modified values or wrote a file. Those operations are unrelated to the abstraction suggested by the function's name.

Separating computation from presentation or external interaction often improves composability:

```
1 result = compute_statistics(data)  
2 display_statistics(result)
```

The first function can then be reused independently of how the result is displayed.

5.9 Documentation and Type Annotations

A function's implementation describes how it performs a computation, but callers primarily need to understand its **contract**.

A docstring can document that contract:

```
1 def mean(values):  
2     """  
3     Compute the arithmetic mean of a non-empty collection.  
4  
5     Inputs
```

```

6  -----
7  values : iterable of numbers
8      Values used in the calculation.
9
10 Outputs
11 -----
12 float
13     Arithmetic mean of the supplied values.
14     """
15     return sum(values) / len(values)

```

Useful documentation explains assumptions, inputs, outputs, side effects, and exceptional conditions when they are not already evident. It should not simply translate each line of implementation into prose.

Python also supports **type annotations**:

```

1 def scale(value: float, factor: float) -> float:
2     return value * factor

```

Annotations communicate intended types to readers and to external tools. They do not, by themselves, impose runtime type checking.

For collections, annotations can describe more structured interfaces:

```

1 def normalise(values: list[float]) -> list[float]:
2     ...

```

and unions can express alternatives:

```

1 def find_value(...) -> int | None:
2     ...

```

Type annotations therefore complement Python's dynamic typing rather than replacing it. Objects still carry their types at runtime, while annotations provide additional information about the intended interface.

A function is most useful when its interface captures a meaningful unit of behaviour while hiding details that the caller does not need to know. This principle of **abstraction** will recur throughout the notes. Data structures expose operations without requiring callers to know their internal representation, and classes extend the same idea by combining persistent state with related behaviour.

6 Numeric Types

Numerical computation requires more than a generic notion of “number”. Integers, real-valued approximations, and complex numbers have different mathematical properties and different computational representations. Python reflects these distinctions through several built-in numeric types.

The principal numeric types are:

Type	Mathematical role	Representation
int	Integers	Arbitrary-precision integer
float	Approximation to real numbers	Fixed-precision floating point
complex	Complex numbers	Pair of floating-point components

These types support many of the same arithmetic operators, but their internal representations and numerical behaviour differ substantially.

6.1 Integers

Python’s int type represents integers:

```
1 count = 42
2 offset = -7
```

Unlike fixed-width integer types commonly found in lower-level languages, Python integers are not restricted to a predetermined range such as $[-2^{31}, 2^{31} - 1]$. Their representation can grow as required by the magnitude of the value, subject to available memory.

For example:

```
1 value = 2 ** 200
```

produces an exact integer rather than an overflow caused by exceeding a fixed machine-word size.

This does not mean that arbitrarily large integers have constant computational cost. Larger values require more memory and, eventually, more work for arithmetic operations. The distinction between the mathematical operation and its computational cost will become important when algorithm analysis is introduced.

6.1.1 Positional Representation

An integer can be represented in any positional numeral system. For a base b , a sequence of digits

$$d_k d_{k-1} \dots d_1 d_0$$

represents the value

$$N = \sum_{i=0}^k d_i b^i,$$

where:

$$0 \leq d_i < b.$$

Decimal notation uses $b = 10$. For example:

$$472 = 4(10^2) + 7(10^1) + 2(10^0) = 400 + 70 + 2.$$

Computers commonly use binary representation, where $b = 2$ and each digit is either 0 or 1. Thus:

$$101101_2 = 1(2^5) + 0(2^4) + 1(2^3) + 1(2^2) + 0(2^1) + 1(2^0),$$

so:

$$101101_2 = 32 + 8 + 4 + 1 = 45_{10}.$$

Python allows integer literals to be written explicitly in several bases:

```
1 binary = 0b101101
2 octal = 0o55
3 decimal = 45
4 hexadecimal = 0x2D
```

All four names refer to integer objects representing the same mathematical value. The notation used in the source code does not create a different numeric type.

This distinction is useful:

numeral representation $\neq$ numeric value.

Binary, octal, decimal, and hexadecimal are different textual representations of integers, not different kinds of integer objects.

6.1.2 Integer Division and Remainders

Division requires particular care because several different operations can be meant by “divide”.

The `/` operator performs **true division**:

```
1 27 / 4
```

and produces:

```
1 6.75
```

The `//` operator performs **floor division**:

```
1 27 // 4
```

which produces:

```
1 6
```

while `%` computes the corresponding remainder:

```
1 27 % 4
```

which produces:

```
1 3
```

These operations are related mathematically. For integers n and m , with $m \neq 0$, define:

$$q = \left\lfloor \frac{n}{m} \right\rfloor$$

and:

$$r = n - qm.$$

Then:

$$\boxed{n = qm + r.} \tag{1}$$

For:

$$n = 27, \quad m = 4,$$

we obtain:

$$q = \left\lfloor \frac{27}{4} \right\rfloor = 6,$$

and therefore:

$$r = 27 - (6)(4) = 3.$$

Hence:

$$27 = (6)(4) + 3.$$

This identity explains the relationship between Python's operators:

```
1 q = n // m
2 r = n % m
```

so that:

```
1 n == q * m + r
```

holds.

The use of the floor is particularly important for negative values. Consider:

$$n = -27, \quad m = 4.$$

Since:

$$-\frac{27}{4} = -6.75,$$

flooring gives:

$$q = \lfloor -6.75 \rfloor = -7,$$

not -6 . The remainder is then:

$$r = -27 - (-7)(4) = 1.$$

Thus:

$$-27 = (-7)(4) + 1.$$

Correspondingly:

```
1 -27 // 4
```

produces -7 , while:

```
1 -27 % 4
```

produces 1 .

Floor division should therefore not be understood simply as “discarding the decimal part”. Truncation toward zero and flooring toward negative infinity are different operations for negative values.

The quotient-remainder relationship appears naturally in many algorithms, including cyclic indexing, divisibility tests, conversion between numeral systems, and hashing.

6.2 Floating-Point Numbers

Integers can be represented exactly, provided sufficient memory is available. Representing arbitrary real numbers is fundamentally different.

Between any two distinct real numbers there are infinitely many other real numbers. A computer with finite memory cannot represent all of them exactly. Numerical computation therefore uses finite approximations.

Python's float type provides such a representation.

Typical floating-point literals include:

```
1 temperature = 21.5
2 probability = 0.125
3 avogadro = 6.022e23
```

The important point is that a floating-point value should not generally be interpreted as an exact real number. It is a representable machine number used to approximate one.

6.2.1 Binary Fractions

The source of many apparently surprising floating-point results becomes clearer by considering positional representation again.

A decimal fraction can be written as:

$$0.d_1d_2d_3\dots = \sum_{i=1}^{\infty} d_i 10^{-i}.$$

For example:

$$0.125 = 1(10^{-1}) + 2(10^{-2}) + 5(10^{-3}).$$

Binary fractions use powers of 2 instead:

$$0.b_1b_2b_3\dots_2 = \sum_{i=1}^{\infty} b_i 2^{-i}.$$

The decimal value 0.125 has a finite binary representation:

$$0.125 = \frac{1}{8} = 2^{-3} = 0.001_2.$$

However, many decimal fractions do not have finite binary representations. The familiar decimal value:

$$0.1 = \frac{1}{10}$$

is one of them.

Since:

$$10 = 2 \cdot 5,$$

the denominator contains a factor other than 2. Consequently, its binary fractional expansion does not terminate. A finite binary floating-point representation must therefore store an approximation to 0.1, not the exact rational number $1/10$.

This is analogous to representing:

$$\frac{1}{3}$$

in decimal notation:

$$0.333333 \dots$$

A finite decimal representation must eventually be rounded.

6.2.2 Floating-Point Representation

A floating-point number separates significant digits from scale. Conceptually, a nonzero value can be represented in the form:

$$x = (-1)^s \times m \times 2^e,$$

where:

- s determines the sign;
- m is the significand;

The exponent determines the scale of the number, while the significand determines its precision.

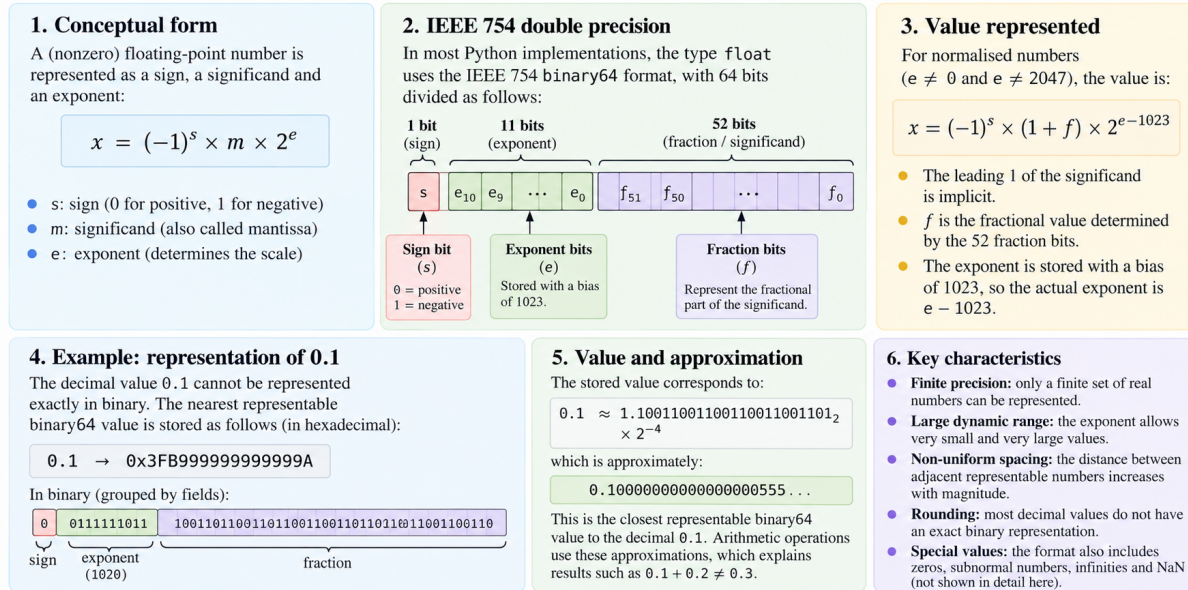


Figure 7: A floating-point representation separates a number into sign, significand, and exponent, allowing a wide range of magnitudes with finite precision.

Python floating-point arithmetic on ordinary implementations uses finite binary precision. Consequently, only a finite subset of the real numbers can be represented directly.

If the exact real value x is not representable, it must be mapped to a nearby machine number:

$$x \longrightarrow \text{fl}(x),$$

where $\text{fl}(x)$ denotes its floating-point representation.

This approximation occurs before subsequent arithmetic is considered.

6.2.3 Representation Error

The effect is visible in a familiar example:

1 $0.1 + 0.2 == 0.3$

which evaluates to:

1 False

This does not indicate an error in addition. The decimal literals themselves are converted to nearby representable binary floating-point values. Arithmetic is then performed on those approximations and rounded to another representable value.

Conceptually:

$$\text{fl}(0.1) + \text{fl}(0.2)$$

need not equal:

$$\text{fl}(0.3)$$

exactly.

Inspecting the result with sufficient decimal precision reveals the difference:

1 `format(0.1 + 0.2, ".17f")`

gives approximately:

1 `0.30000000000000004`

The error is very small, but exact equality is a strict condition.

This is why floating-point results should generally not be compared for exact equality when rounding error may be present.

6.2.4 Comparing Floating-Point Values

Suppose a computation produces a and another produces b . Instead of requiring:

$$a = b,$$

we can ask whether the difference is sufficiently small:

$$|a - b| \leq \varepsilon,$$

where ε is a tolerance appropriate to the problem.

An absolute tolerance alone can be inadequate when values vary greatly in magnitude. A difference of 10^{-6} may be negligible when comparing values near 10^6 but substantial when comparing values near 10^{-7} .

A more useful comparison combines relative and absolute tolerances. Conceptually, two values may be considered sufficiently close when:

$$|a - b| \leq \max(\varepsilon_{\text{rel}} \max(|a|, |b|), \varepsilon_{\text{abs}}). \quad (2)$$

Python's `math.isclose` provides this form of comparison:

```
1 import math
2
3 math.isclose(a, b, rel_tol=1e-9, abs_tol=1e-12)
```

The tolerances should reflect the numerical problem rather than being chosen merely to force a desired result.

6.2.5 Machine Precision

Because floating-point numbers are discrete, representable values have gaps between them. Near 1, there is a smallest representable value greater than 1.

Machine epsilon is commonly used to describe this scale. Informally, it characterises the separation between 1 and the next larger representable floating-point number.

Python exposes information about its floating-point representation through:

```
1 import sys
2
3 sys.float_info
```

including the value:

```
1 sys.float_info.epsilon
```

For the usual binary64 representation, this is approximately:

$$\varepsilon_{\text{machine}} \approx 2.22 \times 10^{-16}.$$

This value should not be interpreted as a universal tolerance for numerical comparisons. It characterises floating-point precision near 1; an appropriate tolerance still depends on the scale and purpose of a computation.

Another important consequence is that floating-point spacing is not constant over the entire number line. As magnitude increases, the distance between adjacent representable values also increases.

Thus floating point provides a trade-off:

large dynamic range with finite relative precision.

6.2.6 Accumulation of Rounding Error

A small representation error may become relevant when an operation is repeated many times.

Consider:

```
1 total = 0.0
2
3 for _ in range(10):
4     total += 0.1
```

Mathematically, the expected result is:

$$10(0.1) = 1.$$

The computed floating-point value, however, need not be exactly 1 because each addition operates on finite approximations.

This does not imply that floating-point arithmetic is unreliable. It means that numerical algorithms must be designed with finite precision in mind.

Python provides `math.fsum` for more accurate summation of floating-point values:

```
1 import math
2
3 total = math.fsum(values)
```

The existence of such algorithms illustrates an important principle: mathematically equivalent formulations need not have equivalent numerical behaviour on a finite computer.

6.3 Numerical Stability

Representation error is unavoidable in finite-precision arithmetic, but algorithms differ in how strongly they amplify that error.

A classic example is subtraction between nearly equal numbers. Suppose:

$$x = 1.234567$$

and:

$$y = 1.234566.$$

The difference is:

$$x - y = 0.000001.$$

Most leading significant digits cancel. If x and y are themselves approximations, the relative error in the small difference can be much larger than the relative errors in the original values. This phenomenon is known as **catastrophic cancellation**.

A more instructive example appears in the quadratic formula. For:

$$ax^2 + bx + c = 0,$$

the roots are:

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}. \quad (3)$$

Mathematically, this formula is exact. Numerically, if:

$$b^2 \gg 4ac$$

and the sign causes two nearly equal quantities to be subtracted, one root may suffer substantial cancellation.

This distinction motivates the concept of **numerical stability**. A numerically stable method is designed so that small errors introduced by finite representation do not grow unnecessarily during the computation.

The general lesson is important:

mathematical equivalence does not guarantee computational equivalence.

This principle will recur in numerical algorithms, optimisation, statistics, and machine learning.

6.4 Complex Numbers

Some computations naturally require numbers beyond the real line. Python provides the built-in complex type for values of the form:

$$z = a + bi,$$

where:

$$i^2 = -1.$$

Python writes the imaginary unit as `j`:

```
1 z = 3 + 4j
```

The real and imaginary components are available as:

```
1 z.real
2 z.imag
```

For:

$$z = 3 + 4i,$$

the complex conjugate is:

$$\bar{z} = 3 - 4i,$$

and can be obtained with:

```
1 z.conjugate()
```

The magnitude, or modulus, is:

$$|z| = \sqrt{a^2 + b^2}.$$

For $z = 3 + 4i$:

$$|z| = \sqrt{3^2 + 4^2} = 5.$$

Python therefore gives:

```
1 abs(3 + 4j)
```

as 5.0.

A complex number can also be represented in polar form. If:

$$r = |z|$$

and θ is its argument, then:

$$z = r(\cos \theta + i \sin \theta).$$

Using Euler's identity:

$$e^{i\theta} = \cos \theta + i \sin \theta,$$

this becomes:

$$\boxed{z = re^{i\theta}.} \tag{4}$$

The rectangular and polar forms emphasise different aspects of the same number:

$$a + bi \quad \longleftrightarrow \quad re^{i\theta}.$$

Python's `cmath` module provides functions designed for complex-valued computation.

Complex numbers arise naturally in applications involving oscillations, Fourier analysis, signal processing, control systems, electrical engineering, computational physics, and quantum mechanics. Their inclusion as a built-in numeric type allows Python to represent these computations directly without constructing a separate numerical abstraction.

The numeric types considered here illustrate a broader principle that will recur throughout the notes: an abstract mathematical object and its computational representation are related, but they are not identical. Understanding both levels is necessary for reasoning about the correctness, limitations, and computational behaviour of a program.

7 Strings

Programs frequently manipulate information that is textual rather than numerical: names, messages, file contents, identifiers, documents, and data received from external systems. Python represents text through the `str` type.

A string is an **immutable sequence of Unicode characters**. This description combines three important properties:

- it is a sequence, so order and position matter;
- its elements represent Unicode text;
- it is immutable, so an existing string cannot be modified after creation.

These properties determine most of the behaviour of strings in Python.

7.1 String Literals

String literals can be delimited by either single or double quotation marks:

```
1 language = "Python"
2 message = 'Data processing complete'
```

The choice does not create different types of strings. It can simply make a literal easier to express when one kind of quotation mark occurs in the text:

```
1 message = "Python's object model"
```

Special characters can be represented with **escape sequences**. For example:

```
1 message = "first line\nsecond line"
```

uses `\n` to represent a newline, while:

```
1 columns = "name\tvalue"
```

uses `\t` to represent a tab.

A backslash itself must normally be escaped:

```
1 path = "C:\\data\\results"
```

When backslashes should be interpreted literally, Python also provides **raw string literals**:

```
1 pattern = r"\d+\.\d+"
```

The prefix `r` changes how backslashes in the literal are interpreted when the string is created. It does not create a different string type.

Triple-quoted strings allow text to span several lines directly:

```
1 message = """Processing complete.  
2 The results are ready for inspection."""
```

They are also used for docstrings, as introduced in the discussion of functions.

7.2 Strings as Sequences

Because a string is a sequence, each character has a position. Python uses zero-based indexing.

For:

```
1 text = "Python"
```

the positions are:

index	0	1	2	3	4	5
character	<i>P</i>	<i>y</i>	<i>t</i>	<i>h</i>	<i>o</i>	<i>n</i>

Therefore:

```
1 text[0]
```

produces:

```
1 'P'
```

and:

```
1 text[5]
```

produces:

```
1 'n'
```

Python has no separate built-in character type. Indexing a string returns another string whose length is one.

Negative indices describe positions relative to the end:

```
1 text[-1]
```

returns the final character, while:

```
1 text[-2]
```

returns the second-to-last character.

For a sequence of length n , the correspondence is:

$$\text{index} - k \longleftrightarrow n - k,$$

whenever the resulting position lies within the sequence.

7.2.1 Slicing

A contiguous part of a string can be selected using slicing:

```
1 text[start:stop]
```

As with range, Python follows the half-open convention:

[start, stop).

The starting position is included and the stopping position is excluded.

For example:

```
1 text = "Python"
2
3 text[1:4]
```

returns:

```
1 'yth'
```

because the selected indices are:

1, 2, 3.

The number of selected elements is therefore simply:

stop — start.

This is one reason the half-open convention is convenient.

Either boundary can be omitted:

```
1 text[:3]
2 text[3:]
```

and a third component specifies a step:

```
1 text[start:stop:step]
```

For example:

```
1 text[::2]
```

selects every second character, while:

```
1 text[::-1]
```

traverses the string in reverse order.

The same indexing and slicing conventions will later apply to other sequence types, particularly lists and tuples.

7.3 String Immutability

Although characters can be accessed by position, they cannot be replaced in an existing string.

For example:

```
1 text = "Python"
2 text[0] = "J"
```

raises an exception.

The reason is not a special restriction on indexing. It follows from the fact that str objects are immutable.

To obtain different text, a new string must be constructed:

```
1 text = "J" + text[1:]
```

The name text is then rebound to the newly created string.

Conceptually:

old string $\longrightarrow$ new string

rather than:

modify existing string.

This is the same distinction between mutation and rebinding introduced earlier.

String methods follow this principle. For example:

```
1 text = "Python"
2
3 lowercase = text.lower()
```

does not modify text. The method produces another string:

```
1 text    -> 'Python'
2 lowercase -> 'python'
```

Immutability makes string values stable once created, but it also has computational consequences when many transformations are performed.

7.4 Concatenation and Repetition

Strings support concatenation with +:

```
1 first = "data"
2 second = "science"
3
4 title = first + " " + second
```

which produces:

```
1 'data science'
```

Repetition is expressed with *:

```
1 separator = "-" * 40
```

These operations produce new strings because existing strings cannot be modified.

This becomes relevant when constructing a large string incrementally. Consider:

```
1 result = ""
2
3 for word in words:
4     result += word
```

Conceptually, each concatenation may require constructing a new string containing the accumulated text. Repeated concatenation can therefore perform substantially more copying than the final string alone would suggest.

When many pieces must be combined, a more appropriate operation is usually:

```
1 result = "".join(words)
```

Here the intended operation is explicit: combine a collection of strings using a separator.

For example:

```
1 fields = ["sensor-01", "21.5", "active"]
2
3 line = ",".join(fields)
```

produces:

```
1 'sensor-01,21.5,active'
```

The difference illustrates a principle that will recur throughout the study of data structures:

an operation's syntax does not by itself reveal its computational cost.

Understanding representation is necessary to reason about efficiency.

7.5 Membership and Comparison

The membership operators `in` and `not in` can be used with strings:

```
1 "data" in "data science"
```

evaluates to True.

The searched value may contain more than one character, so string membership can express substring search as well as single-character membership.

Strings also support lexicographic comparison:

```
1 "algorithm" < "array"
```

Comparison proceeds according to the ordering of the underlying characters, examining corresponding positions until a difference determines the result.

Conceptually, for two strings:

$$S = s_0 s_1 \dots s_{n-1}$$

and:

$$T = t_0 t_1 \dots t_{m-1},$$

the first position k for which:

$$s_k \neq t_k$$

determines their order. If no such position exists within the shorter string, the shorter sequence precedes the longer one.

This same notion of lexicographic ordering applies more generally to ordered sequences.

7.6 String Transformations

Python provides many string methods, but they are easier to understand when grouped by purpose rather than memorised individually.

Case transformations include:

```
1 text.lower()  
2 text.upper()  
3 text.title()
```

Whitespace can be removed from the boundaries of a string with:

```
1 text.strip()
2 text.lstrip()
3 text.rstrip()
```

Search and replacement operations include:

```
1 text.find(substring)
2 text.startswith(prefix)
3 text.endswith(suffix)
4 text.replace(old, new)
```

Strings can also be classified according to their contents:

```
1 text.isalpha()
2 text.isdigit()
3 text.isalnum()
4 text.isspace()
```

These methods answer questions about a string or construct a transformed string; they do not modify the original object.

Two particularly important operations are **splitting** and **joining**.

A string can be decomposed into pieces:

```
1 record = "sensor-01,21.5,active"
2
3 fields = record.split(",")
```

producing:

```
1 ["sensor-01", "21.5", "active"]
```

Conversely:

```
1 record = ",".join(fields)
```

constructs a string from those pieces.

The two operations therefore move between two useful representations:

structured sequence of strings $\longleftrightarrow$ single textual representation.

This pattern appears frequently when processing files, delimited records, logs, and other textual data.

7.7 String Formatting

Programs often need to construct text from values of different types. Python's formatted string literals, or **f-strings**, provide a direct mechanism:

```
1 sensor = "A17"
2 temperature = 21.537
3
4 message = f'Sensor {sensor}: {temperature:.2f} C'
```

which produces:

```
1 'Sensor A17: 21.54 C'
```

The expression inside `{}` is evaluated and its value is incorporated into the resulting string.

Formatting specifications can control details such as precision, alignment, width, and numeric representation:

```
1 value = 0.87342
2
3 message = f'Accuracy: {value:.2%}'
```

producing:

```
1 'Accuracy: 87.34%'
```

Formatting should be distinguished from the underlying value. The floating-point object has not changed because it is displayed with two decimal places. Formatting constructs a textual representation of that value.

Thus:

value $\neq$ textual representation of the value.

This distinction is important when reading and writing data: text may represent a number, but it remains text until it is parsed into a numeric object.

For example:

```
1 value = float("21.5")
```

converts textual data into a floating-point object, while:

```
1 text = f"{value:.1f}"
```

constructs text from that numeric object.

7.8 Unicode Text

Text processing becomes more subtle when programs must represent languages and symbols beyond a small character set. Python strings use **Unicode**, which provides a common system for identifying textual characters.

A Unicode **code point** is an integer associated with a character. Python provides `ord` to obtain the code point of a character:

```
1 ord("A")
```

and `chr` for the inverse operation:

```
1 chr(65)
```

Thus:

character $\longleftrightarrow$ Unicode code point.

A Python string should therefore be understood as text expressed through Unicode characters, rather than as a sequence of raw bytes.

This distinction becomes important when text crosses the boundary between a Python program and an external representation such as a file or network message.

7.9 Text and Bytes

Computers ultimately store and transmit information as bytes. Unicode text therefore needs an **encoding** when it is converted into a byte representation.

Python distinguishes these concepts explicitly:

```
1 str -> Unicode text
2 bytes -> binary data
```

An encoding defines how Unicode characters are represented as bytes. UTF-8 is a widely used encoding.

A string can be encoded:

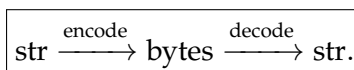
```
1 text = "café"
2
3 data = text.encode("utf-8")
```

The result is a bytes object rather than another string.

The inverse operation is decoding:

```
1 text = data.decode("utf-8")
```

The relationship is:



The encoding must be known because the same byte sequence can have different interpretations under different encoding schemes, and not every byte sequence is valid under every encoding.

This distinction is especially important for file and network I/O. A program may reason about characters and strings internally, while external systems store or transmit encoded bytes.

Confusing text with its encoded representation is a common source of errors. The two abstractions should therefore remain separate:

text is a sequence of characters;	encoded data is a sequence of bytes.
-----------------------------------	--------------------------------------

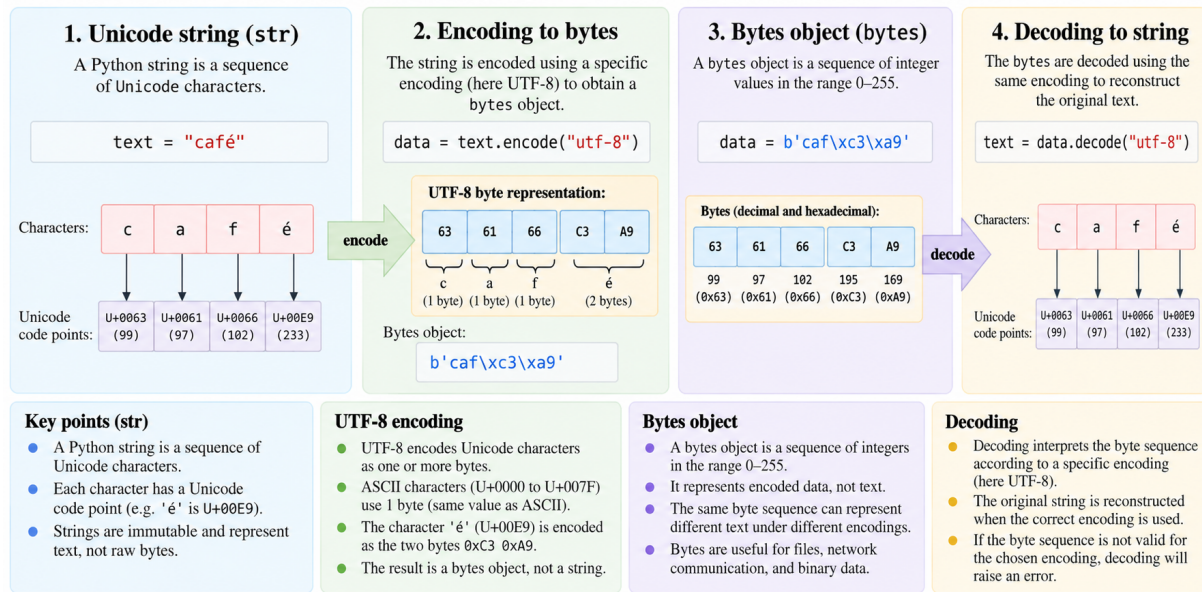


Figure 8: Encoding converts Unicode text into a byte representation, while decoding interprets bytes according to an encoding to reconstruct text.

Strings provide the first major example of a **sequence abstraction** in Python. Their indexing, slicing, iteration, membership, and ordering behaviour will reappear in other sequence types. The next step is to examine Python's built-in containers, where mutability, references, ordering, uniqueness, and key-based access lead to different ways of organising collections of objects.

8 References

- [1] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, *Data structures and algorithms in python*. Wiley Hoboken, NJ, USA, 2013.